

Object-Oriented Programming I

Bit by Bit Coding - Term 2 | Week 7

Key Concepts

- A class is a blueprint; an object (instance) is built from it.
- `__init__` is the constructor - runs automatically when an object is created.
- `self` refers to the current object being worked on.
- Instance attributes belong to each object; class attributes are shared by all.
- Methods are functions defined inside a class.
- A leading underscore (`_name`) signals "private" - a convention, not enforced.

Syntax Reference

Defining a Class & Creating Objects

```
class Player:
    species = "human"          # class attribute (shared)

    def __init__(self, name, hp):
        self.name = name       # instance attribute
        self.hp = hp
        self._secret = "cheat" # "private" by convention

    def take_damage(self, amount):
        self.hp -= amount

p1 = Player("Aria", 100)
p1.take_damage(20)
print(p1.name, p1.hp)        # Aria 80
print(p1.species)           # human
```

Instance vs Class Attributes

- Instance attributes: set in `__init__` with `self`. (unique per object).
- Class attributes: defined in the class body (shared by all objects).
- Access via `self`. or `ClassName`. (e.g., `Player.species`).

Common Patterns

- Store each object's data as instance attributes in `__init__`.
- Add behaviour with methods that read or change `self` attributes.
- Make a new object: `obj = ClassName(args)`.

Tips & Common Mistakes

- Always include `self` as the first parameter of methods.
- `__init__` takes `self` plus the creation args (no `self` when calling).
- Forgetting `self` stops a method from reaching the object's data.
- `_name` is only a convention - Python doesn't truly block access.